COVE Analysis Exercise: Member Call Transcript Review

Author: Zara Z

Here is my plan for this analysis:

1. Clean the data and address as many data quality issues as possible
2. Export a manual-review file for items that do not pass quality checks and cannot be included in the model
3. Use exploratory data analysis (EDA) to identify the themes in the dataset
4. Check those themes against the kinds of problems UnitedHealthcare's own COVE team has publicly said it watches for based on article I found
5. Score each call's sentiment using a model built for conversational text, and corroborate it with a second, independent piece of text for each call.
6. Dig into the primary finding(s)
7. Summarize findings and recommendations

1. Setup

This notebook was created in Google Colab. If you plan on running it yourself, you'll need to upload `COVE Analysis Exercise.xlsx` when prompted (code chunk #4). I've also added as many comments and notes along the way to explain my thought process and selections

In [1]:
```
# Openpyxl is a small library for reading and writing Excel files
# Transformers is needed for the sentiment model in Section 6

!pip install -q openpyxl transformers --upgrade
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/11.7 MB ? eta -:--:--
━━╸━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/11.7 MB 33.5 MB/s eta 0:00:01
━━━━━━━╸━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.0/11.7 MB 68.6 MB/s eta 0:00:01
━━━━━━━━━━━━━━━━━━━━━━━╸━━━━━━━━━━━━━━━ 9.1/11.7 MB 81.5 MB/s eta 0:00:01
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╸ 11.7/11.7 MB 98.8 MB/s eta 0:00:01
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╸ 11.7/11.7 MB 98.8 MB/s eta 0:00:01
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.7/11.7 MB 63.2 MB/s eta 0:00:00
```

In [2]:
```
import re
import io
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer, ENGLISH_STOP_WORDS
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

```
from scipy import stats
import openpyxl
from openpyxl.styles import Font, PatternFill, Alignment
```

In [3]:
```
# Parameters used throughout this analysis are collected here

CONFIG = {
    # Column names from the raw export
    "text_column": "Long Transcript",
    "summary_column": "Call-intent auto summarization",
    "sentiment_column": "Call Sent",
    "category_column": "ACC-1",
    "date_column": "Call date",
    "lob_column": "LOB",
    "state_column": "Mem State",

    # Words to ignore when building themes, on top of standard English stop words l
    "custom_stopwords": {
        "united", "healthcare", "health", "care", "calling", "call", "thank",
        "thanks", "hello", "hi", "good", "morning", "afternoon", "evening",
        "name", "speaking", "pleasure", "today", "yes", "no", "okay", "ok",
        "um", "uh", "advocate", "adv", "caller", "xxxx", "xxxxx", "member",
        "just", "going", "know", "like", "right", "got", "let", "sure",
        "moment", "second", "minute", "verify", "verifying", "verified",
        "automated", "system", "recorded", "quality", "training", "purposes",
        "redacted", "phone", "dob", "ssn", "memberid",
        # Added after reviewing top terms per cluster
        "umm", "mhm", "alright", "said", "told", "did", "didn", "want",
        "think", "remember", "make", "help", "called", "new", "day", "time",
        "week", "ll", "ve", "re", "don", "isn", "won", "maybe",
        "hmm", "mm", "yeah", "ma", "ago", "probably", "correct", "trying",
        "believe", "regarding", "directly", "great", "sounds", "come", "went",
        "say", "says", "saying", "mean", "getting", "sorry", "thing", "doing",
        "tell", "really", "ahead", "able", "appreciate", "guess", "gave", "way",
        "guys", "supposed", "check", "huh", "fine", "talk", "talked", "bye",
        "wanted", "kind", "actually", "gonna", "look", "does", "having",
        "needed", "needs", "somebody", "understand", "stuff", "hear", "little",
        "contact", "using", "reason", "far", "lot", "took", "birth", "id",
        "haven", "people","god", "honey", "nice", "wonderful", "perfect", "definite
        "exactly", "figure", "spoke", "wondering", "mmm", "righty", "telling", "ask
    },

    # Only searching group counts between 8 and 11. Below 8, distinct problems star
    # one group, for example wheelchairs and medication ending up together. Above 1
    # started fragmenting into pieces too thin to reliably read, some with only 2 c
    "k_range": range(8, 12),
    "random_state": 42,

    # Which emotion labels from the sentiment model count as "frustrated" for this
    "frustration_labels": {"anger", "annoyance", "disapproval", "disappointment", "

    # Sentiment model settings
    "chunk_size_words": 70,    # Maximum number of words per chunk sent to model
    "batch_size": 16,          # Number of chunks scored together in a single pass (
}
```

In [4]:
```
# Load the file using Colab's upload widget if available, or a local file otherwise
try:
    from google.colab import files
    print("Upload 'COVE Analysis Exercise.xlsx' now:")
    uploaded = files.upload()
    SRC = list(uploaded.keys())[0]
except ImportError:
    SRC = "COVE Analysis Exercise.xlsx" # Make sure file name matches this
    print(f"Not running in Colab. Looking for a local file named '{SRC}'.")

print("Using file:", SRC)
```

```
Upload 'COVE Analysis Exercise.xlsx' now:
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving COVE Analysis Exercise.xlsx to COVE Analysis Exercise.xlsx
Using file: COVE Analysis Exercise.xlsx
```

2. Data cleaning

1. **2 rows with no transcripts at all.** These are summary-only rows. Rather than dropping them, I think it's safe to fall back to a shorter auto-generated summary of the call for those two rows, so we're keeping as much as possible of the data for a more complete analysis.
2. **Unprotected personal information in the transcript text.** A handful of transcripts still show a phone number, date of birth, or name, even though the structured `Mem First` and `Mem Last` columns had already been anonymized. I will redact this information below, we should not need them. Also, HIPAA? I could be wrong, please correct me if so :)

In [5]:
```
def load_raw(path):
    wb = openpyxl.load_workbook(path, data_only=True)
    ws = wb["Sheet1"]                                # The data lives on a sheet name
    headers = [c.value for c in ws[1]][:15]          # Export has 15 columns
    rows = [[ws.cell(row=r, column=c).value for c in range(1, 16)] for r in range(2
    df = pd.DataFrame(rows, columns=headers)
    df.insert(0, "row_num", range(2, 2 + len(df)))  # Keeps the original Excel row
    return df

raw = load_raw(SRC)
print("Raw shape:", raw.shape)
raw.head(3)
```

```
Raw shape: (100, 16)
```

Out[5]:

	row_num	Call date	Call-intent auto summarization	Long Transcript	Call Sent	Mem First	Mem Last	Mem State
0	2	2026-05-01	"The caller is seeking clarification regarding...	"Adv: Good afternoon. Thanks for calling Unite...	-5.1	Patient_01_First	Patient_01_Last	VA
1	3	2026-05-01	"The caller inquired about returning unused ox...	"Adv: Hi, thank you so much for calling United...	0.2	Patient_02_First	Patient_02_Last	TN
2	4	2026-05-01	"The caller inquired whether the insurance pla...	"Caller: She ever come back here? \nAdv: Hello,...	-0.2	Patient_03_First	Patient_03_Last	NC

In [6]:
```
# Count duplicate rows only if they have matching transcript text. This guards
# against pandas treating two blank transcripts as equal to each other.
text_col = CONFIG["text_column"]
has_text = raw[text_col].notna() & (raw[text_col].astype(str).str.strip() != "")
dupe_mask = has_text & raw.duplicated(subset=[text_col], keep="first")

print("True duplicate rows found:", raw.loc[dupe_mask, "row_num"].tolist() or "None
df = raw.loc[~dupe_mask].reset_index(drop=True)
print("Shape after removing duplicates:", df.shape)

df["has_transcript"] = df[text_col].notna() & (df[text_col].astype(str).str.strip()
print("Rows with no transcript text, summary-only:", df.loc[~df["has_transcript"],
```

```
True duplicate rows found: None.
Shape after removing duplicates: (100, 16)
Rows with no transcript text, summary-only: [88, 97]
```

Redacting personal information

I am working off of the assumption that Personal Health Information (PHI) should not appear in text that gets analyzed. I will be using pattern matching, known as regular expressions or regex, to find and remove phone numbers, dates of birth, SSNs, and member ID numbers from the transcript text. I did not attempt to remove full names, since regex cannot reliably tell a person's name apart from an ordinary word; that requires a more sophisticated language tool than I am using here. I did find at least two name mentions while scanning

through transcripts, and I am noting that limitation here rather than claiming this redaction step caught everything.

In [7]:
```
# Regex for phone numbers, dates of birth, SSNs, and member ID numbers
PHONE_RE = re.compile(r'\b(?:1[-.\s]?)?\(?\d{3}\)?[-.\s]\d{3}[-.\s]\d{4}\b')   # Fo
DOB_RE = re.compile(r'\b(0[1-9]|1[0-2])[/-](0[1-9]|[12]\d|3[01])[/-](19|20)\d{2}\b'
SSN_RE = re.compile(r'\b\d{3}-\d{2}-\d{4}\b')
MEMBERID_RE = re.compile(r'\bmember (?:id|ID)(?: number)?\s*(?:is\s*)?\d{6,12}\b',

def redact_phi(text):
    if not isinstance(text, str):
        return text
    text = SSN_RE.sub("[REDACTED-SSN]", text)
    text = DOB_RE.sub("[REDACTED-DOB]", text)
    text = PHONE_RE.sub("[REDACTED-PHONE]", text)
    text = MEMBERID_RE.sub("[REDACTED-MEMBERID]", text)
    return text

summary_col = CONFIG["summary_column"]
for col in [summary_col, text_col]:
    df[col] = df[col].fillna("").astype(str).str.strip().str.strip('"')  # Handling

df["transcript_redacted"] = df[text_col].apply(redact_phi)
df["summary_redacted"] = df[summary_col].apply(redact_phi)
df["analysis_text"] = df["transcript_redacted"].where(df["has_transcript"], df["sum
df["phi_found"] = df[text_col] != df["transcript_redacted"]

print(f"Personal information redacted in {df['phi_found'].sum()} of {len(df)} rows:

# Sanity check on one row
example = df[df["row_num"] == 12].iloc[0]
before_sents = re.split(r'(?<=[.\n])', example[text_col])
after_sents = re.split(r'(?<=[.\n])', example["transcript_redacted"])
print("\nRow 12, sentences where redaction changed the text:\n")
for b, a in zip(before_sents, after_sents):
    if b != a:
        print("BEFORE:", b.strip())
        print("AFTER :", a.strip())
        print()
```

```
Personal information redacted in 11 of 100 rows: [12, 30, 34, 43, 48, 54, 59, 64, 8
4, 94, 101]

Row 12, sentences where redaction changed the text:

BEFORE: Adv: 11/23/1954.
AFTER : Adv: [REDACTED-DOB].

BEFORE: I have two phone numbers, so I'm going to give you the mobile phone number t
hat I have on file, 909-269-5193.
AFTER : I have two phone numbers, so I'm going to give you the mobile phone number t
hat I have on file, [REDACTED-PHONE].

BEFORE: Adv: Mary B as in boy Louis 11/23/1954 is Earth.
AFTER : Adv: Mary B as in boy Louis [REDACTED-DOB] is Earth.
```

Works great! Moving on..

The existing category tags are incomplete

In [8]:
```
cat_col = CONFIG["category_column"]
untagged_mask = df[cat_col].isna() | df[cat_col].isin(["INCOMPLETE CALL DATA", "NEX
n_untagged = int(untagged_mask.sum())
n_tagged = len(df) - n_untagged

print(f"{n_tagged}/{len(df)} calls, or {n_tagged/len(df)*100:.0f}%, have a usable {
      f"{n_untagged} calls, or {n_untagged/len(df)*100:.0f}%, do not.")
```

```
78/100 calls, or 78%, have a usable ACC-1 tag. 22 calls, or 22%, do not.
```

In [9]:
```
# Checking whether ACC-2 or ACC-Con ever rescue a call that ACC-1 leaves untagged,
# are tag columns but the check above only looked at ACC-1
tag_cols = ["ACC-1", "ACC-2", "ACC-Con"]
placeholder_vals = ["INCOMPLETE CALL DATA", "NEXIDIA TRANSCRIPT UNAVAILABLE"]

all_null_together = (df["ACC-1"].isna() == df["ACC-2"].isna()).all() and (df["ACC-1
print(f"ACC-1, ACC-2, and ACC-Con are missing on exactly the same rows: {all_null_t

rescue_mask = untagged_mask & df["ACC-2"].notna() & ~df["ACC-2"].isin(placeholder_v
print(f"Calls untagged by ACC-1 where ACC-2 has a real value ACC-1 doesn't: {rescue

plan_leak_mask = df["ACC-Con"].astype(str).str.contains(r"Medicare Advantage|\(HMO|
print(f"Rows where ACC-Con actually holds what looks like a plan name instead of a
df.loc[plan_leak_mask, ["row_num", "ACC-1", "ACC-2", "ACC-Con"]]
```

```
ACC-1, ACC-2, and ACC-Con are missing on exactly the same rows: True.
Calls untagged by ACC-1 where ACC-2 has a real value ACC-1 doesn't: 0.
Rows where ACC-Con actually holds what looks like a plan name instead of a tag: 3.
```

Out[9]:

	row_num	ACC-1	ACC-2	ACC-Con
66	68	INCOMPLETE CALL DATA	INCOMPLETE CALL DATA	AARP Medicare Advantage from UHC NC-0021 (HMO-...
82	84	RX REFILL DENIED	RX OVERRIDE REQUEST	AARP Medicare Advantage from UHC IN-0006 (PPO)
83	85	BENEFITS COVERAGE BREAKDOWN	BENEFITS DME OR SUPPLIES COVERED	AARP Medicare Advantage Essentials from UHC IN...

ACC-1, ACC-2, and ACC-Con turn out to be a three-level hierarchy, a broad category, a sub-category, and a specific reason, that gets filled in together or not at all, rather than three independent tagging attempts. They are missing on the exact same rows every time, so combining them does not shrink the 22% gap from the previous check, a call either has all three levels or none of them.

What combining them does give me is a fuller label for the calls that are already tagged, for example "BENEFITS" alone becomes "BENEFITS > BENEFITS COVERAGE BREAKDOWN > BENEFITS DME OR SUPPLIES COVERED." I built that combined field below.

In [10]:
```
# ACC-1, ACC-2, and ACC-Con combined into one fuller label for calls that have them
df["category_full"] = df[tag_cols].apply(lambda r: " > ".join(r.dropna()), axis=1)
df["category_full"] = df["category_full"].where(df["category_full"] != "", "Untagge

df.loc[~untagged_mask, "category_full"].value_counts().head(10)
```

Out[10]:

category_full	count
BENEFITS > BENEFITS COVERAGE BREAKDOWN > BENEFITS DME OR SUPPLIES COVERED	11
TRANSPORTATION > TRANSPORTATION CHANGE OR SCHEDULE OR CANCEL TRIP > INFO GIVEN OR ISSUE RESOLVED	8
BENEFITS > ESTIMATION OF OOP COST > BENEFITS DME OR SUPPLIES COVERED	6
RX INQUIRY > RX REFILL DENIED > RX OVERRIDE REQUEST	5
RX INQUIRY > RX COVERAGE DETERMINATION > RX NOT COVERED	3
PROVIDER SEARCH > FIND A PROVIDER NEAR ME > DME	3
CLAIMS INQUIRY > UNDERSTANDING PATIENT RESPONSIBILITY OR EOB > COPAY OR COINSURANCE OR DEDUCTIBLE	2
PRIOR AUTH INQUIRY > PRIOR AUTH STATUS CHECK > PRIOR AUTH NOT RECEIVED	2
RX INQUIRY > RX PRIOR AUTH > RX AUTH NOT OBTAINED OR NOT REQUIRED	2
CLAIMS INQUIRY > WHY WAS MY CLAIM DENIED OR PARTIALLY DENIED > CLAIM NO COVERAGE AT TIME OF SERVICE	2

dtype: int64

In [11]:
```
# Adding the same placeholder exclusion used above, since a call tagged "INCOMPLETE
# ACC-1 still joins into a non-empty category_full string and would otherwise slip
df.loc[df["ACC-1"].isin(placeholder_vals), "category_full"] = "Untagged"

untagged_mask_combined = df["category_full"] == "Untagged"
n_untagged_combined = int(untagged_mask_combined.sum())
n_tagged_combined = len(df) - n_untagged_combined

print(f"Using ACC-1, ACC-2, and ACC-Con combined instead: {n_tagged_combined}/{len(
      f"{n_tagged_combined/len(df)*100:.0f}%, have a usable tag. {n_untagged_combin
      f"{n_untagged_combined/len(df)*100:.0f}%, still do not.")
```

```
Using ACC-1, ACC-2, and ACC-Con combined instead: 78/100 calls, or 78%, have a usabl
e tag. 22 calls, or 22%, still do not.
```

This gap is still large enough for me to not completely trust the category tag alone. This is why I will be identifying themes independently instead.

3. Manual review workbook

I exported every row I touched or flagged into its own tab in a separate Excel file: the transcripts with no text, the redacted PHI shown before and after, and every call the platform left uncategorized. Anything unusual is in this file for an analyst to check by hand as needed.

In [12]:
```
HEADER_FILL = PatternFill(start_color="2A78D6", end_color="2A78D6", fill_type="soli
HEADER_FONT = Font(bold=True, color="FFFFFF")
WRAP = Alignment(wrap_text=True, vertical="top")

def _write_sheet(wb, name, sheet_df, col_widths=None, wrap_cols=None):
    # Writes a dataframe into a new sheet with a blue header row and wrapped text w
    ws = wb.create_sheet(name)
    ws.append(list(sheet_df.columns))
    for cell in ws[1]:
        cell.fill = HEADER_FILL
        cell.font = HEADER_FONT
    for row in sheet_df.itertuples(index=False):
        ws.append(list(row))
    wrap_cols = wrap_cols or set()
    col_widths = col_widths or {}
    for i, col in enumerate(sheet_df.columns, start=1):
        letter = ws.cell(row=1, column=i).column_letter
        ws.column_dimensions[letter].width = col_widths.get(col, 18)
        if col in wrap_cols:
            for cell in ws[letter][1:]:
                cell.alignment = WRAP
    ws.freeze_panes = "A2"   # Keeps the header row visible while scrolling
    return ws

wb_out = openpyxl.Workbook()
wb_out.remove(wb_out.active)   # Removes the default empty sheet

# README tab
ws = wb_out.create_sheet("README")
ws.column_dimensions["A"].width = 100
readme_lines = [
    "COVE Analysis Exercise: Data Quality and Manual Review Log", "",
    "Every row the cleaning process modified or flagged, for independent review.","
    "Sheets: Missing_Transcript, PHI_Redacted, Untagged_Or_Incomplete.",
]
for i, line in enumerate(readme_lines, start=1):
    cell = ws.cell(row=i, column=1, value=line)
    if i == 1:
        cell.font = Font(bold=True, size=13)
    cell.alignment = Alignment(wrap_text=True, vertical="top")

# Rows with no transcript
df_missing = df.loc[~df["has_transcript"],
                    ["row_num", "UCID", CONFIG["date_column"], CONFIG["state_colum
                     CONFIG["summary_column"], "ACC-1", "ACC-2", "ACC-Con"]].copy(
_write_sheet(wb_out, "Missing_Transcript", df_missing,
             col_widths={CONFIG["summary_column"]: 70}, wrap_cols={CONFIG["summary_

# Redacted rows, showing the original text next to the redacted text
df_phi = df.loc[df["phi_found"], ["row_num", "UCID", CONFIG["date_column"], CONFIG[
df_phi["Original transcript"] = df.loc[df["phi_found"], text_col]
df_phi["Redacted transcript"] = df.loc[df["phi_found"], "transcript_redacted"]
_write_sheet(wb_out, "PHI_Redacted", df_phi,
             col_widths={"Original transcript": 70, "Redacted transcript": 70},
             wrap_cols={"Original transcript", "Redacted transcript"})
```

```
# Calls the platform left uncategorized
df_untagged = df.loc[untagged_mask,
                     ["row_num", "UCID", CONFIG["date_column"], CONFIG["state_colu
                      CONFIG["summary_column"], "ACC-1", "ACC-2", "ACC-Con"]].copy
_write_sheet(wb_out, "Untagged_Or_Incomplete", df_untagged,
             col_widths={CONFIG["summary_column"]: 70}, wrap_cols={CONFIG["summary_

EXPORT_PATH = "COVE_Data_Quality_Review.xlsx"
wb_out.save(EXPORT_PATH)
print("Wrote", EXPORT_PATH)

# Downloads the file automatically in Colab, otherwise it stays in the working dire
try:
    from google.colab import files as colab_files
    colab_files.download(EXPORT_PATH)
except ImportError:
    pass
```

```
Wrote COVE_Data_Quality_Review.xlsx
```

4. Exploratory data analysis: finding themes in the calls

Exploratory data analysis (EDA) means systematically looking at what is actually in the data before drawing conclusions from it. Here, that means figuring out what members are really calling about, rather than trusting category tags alone, since the previous section showed that 22% of calls do not have one.

To do this, I used two standard text-analysis techniques. The first, called TF-IDF, converts each call's text into a set of numbers that highlight the words that matter most in that call, while ignoring common words that appear everywhere. The second, called K-Means, is a method for automatically grouping/ clustering similar calls together based on those numbers, without me telling it what the groups should be in advance. I only look at the caller's own words for this step, not the advocate's language, since phrases like "thank you for calling" would swamp every cluster.

K-Means needs to be told how many groups to look for. I tried a range of group counts and picked the one with the best silhouette score, which is a standard measure of how cleanly separated the resulting groups are from each other. A higher silhouette score means the groups are more distinct, which is better.

In [13]:
```
def extract_caller_lines(transcript):
    # Pulls out just the caller's lines from a transcript formatted with "Adv:" and
    if not isinstance(transcript, str) or "Caller:" not in transcript:
        return transcript   # No "Adv:"/"Caller:" labels here, for example on summa

    lines = transcript.split("\n")
    caller_lines = [l.split("Caller:", 1)[1].strip() for l in lines if "Caller:" in
    return " ".join(caller_lines)
```

```
df["caller_text"] = df.apply(
    lambda r: extract_caller_lines(r[text_col]) if r["has_transcript"] else r["anal
    axis=1,
)

# In case the extraction above came back empty for some row, fall back to whatever
empty = df["caller_text"].str.strip() == ""
df.loc[empty, "caller_text"] = df.loc[empty, "analysis_text"]
```

In [14]:
```
# Two extra cleanup passes on the caller's words specifically, on top of the stopwo
# Appointment times and dates ("11", "30", "2nd") are call-specific, so they'll get
# stripped rather than treated as topic words. Speech disfluencies transcribed lite
# tried", "mm hmm") get collapsed, since they would otherwise look like meaningful
df["caller_text"] = df["caller_text"].str.replace(r"\b\d+(st|nd|rd|th)?\b", "", reg
df["caller_text"] = df["caller_text"].str.replace(r"\b(\w+)[,\s]+\1\b", r"\1", rege

STOPWORDS = list(CONFIG["custom_stopwords"].union(ENGLISH_STOP_WORDS))

vectorizer = TfidfVectorizer(
    max_df=0.6,               # Ignores words that show up in over 60% of calls
    min_df=2,                 # Ignores words that only appear once across the whol
    stop_words=STOPWORDS,
    ngram_range=(1, 2),       # Looks at single words and two word phrases, so it c
)
X = vectorizer.fit_transform(df["caller_text"])

# Tries a handful of group counts, within the range explained in CONFIG above, and
# whichever one has the best silhouette score, which again, means they're more dist
scores = {}
models = {}
for k in CONFIG["k_range"]:
    km = KMeans(n_clusters=k, random_state=CONFIG["random_state"], n_init=10)
    labels = km.fit_predict(X)
    scores[k] = silhouette_score(X, labels)
    models[k] = (km, labels)

best_k = max(scores, key=scores.get)
best_model, best_labels = models[best_k]
df["theme_cluster"] = best_labels

print("Silhouette score by number of clusters:", {k: round(v, 3) for k, v in scores
print(f"\nChosen number of clusters: {best_k}, with a silhouette score of {scores[b
```

```
Silhouette score by number of clusters: {8: np.float64(0.011), 9: np.float64(0.01),
10: np.float64(0.016), 11: np.float64(0.014)}

Chosen number of clusters: 10, with a silhouette score of 0.016.
```

These silhouette scores are fairly low. I went through three rounds of reviewing the actual top terms per group and adding conversational filler, transcription artifacts, and interjections to the stopword list; I also stripped call-specific numbers (times, dates) and collapsed literal word repeats from the transcript before vectorizing. I am treating this grouping as a starting point to set themes.

In [15]:
```
# The words that define each cluster
terms = np.array(vectorizer.get_feature_names_out())
print("Top terms per group:\n")
for c in range(best_k):
    center = best_model.cluster_centers_[c]
    top_idx = center.argsort()[::-1][:12]   # The 12 words that weigh most heavily
    n = (df["theme_cluster"] == c).sum()
    print(f"Group {c} (n={n}): {', '.join(terms[top_idx])}")
```

```
Top terms per group:

Group 0 (n=3): insulin, diabetes, pre, authorization, pump, medication, message, hus
band, error, prior authorization, pre authorization, prior
Group 1 (n=15): medication, doctor, pharmacy, card, pcp, plus, form, plan, address,
nebulizer, prescription, holding
Group 2 (n=4): catheters, smaller, carry, urinary, catheter, charges, submitted, med
icine, stephanie, shouldn, denied, rock
Group 3 (n=14): chair, home, lift, lift chair, doctor, prior, covered, wheelchair, t
herapy, physical therapy, physical, authorization
Group 4 (n=15): synapse, machine, supplies, cpap, mask, order, company, synops, prob
lem, send, sent, doctor
Group 5 (n=13): pay, scooter, equipment, companies, doctor, mobility, walker, nebuli
zer, miss, breathe, company, money
Group 6 (n=6): maria, line, representative, authorized representative, authorized, s
afe, cinco, fully, better, home, apria, nurse behalf
Group 7 (n=12): oxygen, portable, synapse, portable oxygen, order, refill, linda, pa
ying, link, insurance, use, problems
Group 8 (n=13): ride, transportation, dialysis, wheelchair, schedule, walker, wednes
day, appointment, june, medical, pick, appointments
Group 9 (n=5): discount, city, billing, insurance, prescription, wouldn, insurance c
ompany, coverage, company, billing insurance, accept insurance, drug
```

Naming the groups

I read a sample of transcripts from each group above and assigned each one a plain-language theme name based on what those calls were actually about. A few of the raw groups above turned out to be two angles on the same underlying issue, for example diabetes-supply prior authorizations and lift-chair prior authorizations, so I rolled those into one shared theme name.

In [16]:
```
# Business-friendly theme name for each cluster, assigned by hand; a few clusters s
# since they turned out to be two angles on the same underlying issue
THEME_ROLLUP = {
    0: "Prior authorization & equipment approval",
    1: "Pharmacy / medication access",
    2: "Billing & coverage denials",
    3: "Prior authorization & equipment approval",
    4: "Oxygen / equipment vendor & delivery failures",
    5: "Mobility equipment billing & claims",
    6: "Authorized representative calls",
    7: "Oxygen / equipment vendor & delivery failures",
    8: "Transportation scheduling",
    9: "Billing & coverage denials",
```

```
}
df["theme_label"] = df["theme_cluster"].map(THEME_ROLLUP).fillna("Other / uncluster

theme_counts = df["theme_label"].value_counts()
theme_counts
```

Out[16]:

theme_label	count
Oxygen / equipment vendor & delivery failures	27
Prior authorization & equipment approval	17
Pharmacy / medication access	15
Transportation scheduling	13
Mobility equipment billing & claims	13
Billing & coverage denials	9
Authorized representative calls	6

dtype: int64

5. Checking against COVE's own "watch list"

UnitedHealthcare has spoken publicly about the COVE team this analysis is for, so I checked my themes against what the team itself says it watches for.

> Christopher Snowbeck, "UnitedHealthcare's mission control targets customer woes to build its brand," *Star Tribune*, April 2026. Also syndicated via InsuranceNewsNet.

The article names five categories the team watches for: call transfers, digital or online portal confusion, tone signals in speech, medical equipment supplier transition confusion, and external factors outside the company's control. I checked each one against this sample, including the supplier category:

In [17]:
```
# Keyword searches used to check each watch-list category below
txt = df[text_col].str.lower().fillna("")

# Each *_mask below is True for a call if its text matches that category's keywords
# to filter df row-by-row in the table below
transfer_mask = txt.str.contains(r'\btransfer', regex=True, na=False)
digital_mask = txt.str.contains(r'\b(?:website|portal|log ?in|the app|online accoun
weather_mask = txt.str.contains(r'\b(?:weather|storm|hurricane|snow|flood|blizzard)
medicare_mask = df[CONFIG["lob_column"]] == "M&R"   # M&R is short for Medicare & R
supplier_mask = txt.str.contains(r'\b(?:supplier|vendor)\b', regex=True, na=False)

checks = pd.DataFrame({
```

```
    "watch-list category": [
        "Call transfer friction",
        "Digital / self-service confusion",
        "External / environmental factors",
        "Medicare (M&R) vs. other plan type, sentiment gap",
        "Any supplier or vendor mention, generic keyword",
    ],
    "calls matched": [transfer_mask.sum(), digital_mask.sum(), weather_mask.sum(),
                      medicare_mask.sum(), supplier_mask.sum()],
    "mean Call Sent, matched": [
        round(df.loc[transfer_mask, CONFIG["sentiment_column"]].mean(), 2),
        round(df.loc[digital_mask, CONFIG["sentiment_column"]].mean(), 2),
        round(df.loc[weather_mask, CONFIG["sentiment_column"]].mean(), 2) if weathe
        round(df.loc[medicare_mask, CONFIG["sentiment_column"]].mean(), 2),
        round(df.loc[supplier_mask, CONFIG["sentiment_column"]].mean(), 2),
    ],
    "mean Call Sent, rest of sample": [
        round(df.loc[~transfer_mask, CONFIG["sentiment_column"]].mean(), 2),
        round(df.loc[~digital_mask, CONFIG["sentiment_column"]].mean(), 2),
        round(df.loc[~weather_mask, CONFIG["sentiment_column"]].mean(), 2) if weath
        round(df.loc[~medicare_mask, CONFIG["sentiment_column"]].mean(), 2),
        round(df.loc[~supplier_mask, CONFIG["sentiment_column"]].mean(), 2),
    ],
})
checks
```

Out[17]:

	watch-list category	calls matched	mean Call Sent, matched	mean Call Sent, rest of sample
0	Call transfer friction	24	0.82	-0.04
1	Digital / self-service confusion	24	0.80	-0.03
2	External / environmental factors	6	0.97	0.12
3	Medicare (M&R) vs. other plan type, sentiment gap	67	0.03	0.46
4	Any supplier or vendor mention, generic keyword	36	0.59	-0.07

Transfers and digital or portal mentions do not read as pain points here; if anything, they skew notably positive. Calls mentioning a transfer average +0.82 versus -0.04 for the rest of the sample (n=24), and calls mentioning the website, portal, or app average +0.80 versus -0.03 (n=24). Medicare (M&R) sentiment runs lower than the other plan type in this sample (0.03 versus 0.46, n=67), loosely matching the article, though this sample is too small to lean on that alone. Weather mentions are too rare to draw a conclusion from (n=6).

The supplier or vendor row is the interesting one, and not in the direction I expected, since Section 4's clustering had already surfaced issues in the "Oxygen / equipment vendor & delivery failures" theme. Calls that simply mention a supplier or a vendor actually skew positive, not negative (+0.59 versus -0.07 for the rest of the sample, n=36). Most of those 36 calls are ordinary "let's get you set up with an equipment supplier" conversations that

resolve without issue; mentioning a supplier is not itself a problem. A generic keyword search cannot tell that apart from transition confusion, so instead of stopping here, I went ahead and tested something more specific in Section 7: does one particular, named vendor correlate with worse sentiment?

6. Sentiment scoring

For each call, I scored how frustrated the caller sounds using a model called SamLowe/roberta-base-go_emotions. This is a version of a language model called RoBERTa. This version is particularly built using Reddit data and has 28 labels, so it is built for more conversational text rather than formal writing. It scores text against a set of emotion labels including anger, annoyance, disapproval, disappointment, disgust, and confusion, which maps directly onto the kind of frustration signals I am looking for in these calls.

Each call is broken into chunks of about 70 words at a time, since the model can only process a limited amount of text at once. I use the highest frustration score across a call's chunks, rather than the average, since a few polite words near the end of a call should not cancel out a complaint earlier in the same call.

I score two independent pieces of text for each call: the transcript itself, and a separate, shorter auto-generated summary of the call that the platform already provides in a different field. If both independently show the same pattern, that is stronger evidence the frustration signal is true, and not just an artifact of how I processed one particular piece of text.

In [18]:
```
# This step downloads model weights the first time it runs and can take a few minut
!pip install -q --upgrade transformers

from transformers import pipeline
import torch

device = 0 if torch.cuda.is_available() else -1   # Uses GPU because we can only be
emotion_clf = pipeline(
    "text-classification",
    model="SamLowe/roberta-base-go_emotions",
    top_k=None,        # Leave as None so it returns a score for every label
    truncation=True,
    device=device,
)

FRUSTRATION_LABELS = CONFIG["frustration_labels"]
CHUNK_SIZE = CONFIG["chunk_size_words"]

def make_chunks(text, chunk_size_words=CHUNK_SIZE):
    # Breaks a piece of text into chunks small enough for the model to process at o
    if not isinstance(text, str) or not text.strip():
        return []
    words = text.split()
    return [" ".join(words[i:i + chunk_size_words]) for i in range(0, len(words), c
```

```
df["_transcript_chunks"] = df["caller_text"].apply(make_chunks)
df["_summary_chunks"] = df["summary_redacted"].apply(make_chunks)

# Combines every chunk, from both text sources and every call, into one list, so th
# score them together in batches. This is for efficiency. Will matter more with lar
flat_chunks, chunk_owner, chunk_source = [], [], []
for idx, chunks in df["_transcript_chunks"].items():
    flat_chunks.extend(chunks)
    chunk_owner.extend([idx] * len(chunks))
    chunk_source.extend(["transcript"] * len(chunks))
for idx, chunks in df["_summary_chunks"].items():
    flat_chunks.extend(chunks)
    chunk_owner.extend([idx] * len(chunks))
    chunk_source.extend(["summary"] * len(chunks))

all_preds = emotion_clf(flat_chunks, batch_size=CONFIG["batch_size"], truncation=Tr
chunk_frustration = [sum(p["score"] for p in preds if p["label"] in FRUSTRATION_LAB

scores_df = pd.DataFrame({"row": chunk_owner, "source": chunk_source, "score": chun
frustration_by_row = scores_df.loc[scores_df["source"] == "transcript"].groupby("ro
summary_frustration_by_row = scores_df.loc[scores_df["source"] == "summary"].groupb

df["frustration_score"] = df.index.map(frustration_by_row).fillna(0.0)
df["summary_frustration_score"] = df.index.map(summary_frustration_by_row).fillna(0
df.drop(columns=["_transcript_chunks", "_summary_chunks"], inplace=True)

pearson_fr = (-df["frustration_score"]).corr(df[CONFIG["sentiment_column"]])
spearman_fr = (-df["frustration_score"]).corr(df[CONFIG["sentiment_column"]], metho
print(f"Transcript-based frustration score vs. the platform's own {CONFIG['sentimen
      f"Pearson correlation={pearson_fr:.3f}, Spearman correlation={spearman_fr:.3f
print("Pearson and Spearman are both standard ways to measure how closely two numbe
print("on a scale from -1 to 1. Spearman only looks at rank order, so it is less th

agreement = df["frustration_score"].corr(df["summary_frustration_score"])
print(f"\nAgreement between the transcript-based score and the summary-based score:
print("A meaningful positive correlation here means two independent pieces of text
print("calls sound frustrated, which is stronger evidence than relying on the trans
```

```
config.json: 100% 1.92k/1.92k [00:00<00:00, 119kB/s]
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOK
EN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated request
s to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downl
oads.
```

```
model.safetensors: reconstructing file: 100% 499MB / 499MB, 38.3MB/s
model.safetensors: downloading bytes: 474MB, 41.9MB/s
Loading weights: 100% 201/201 [00:00<00:00, 7221.77it/s]
tokenizer_config.json: 100% 380/380 [00:00<00:00, 26.5kB/s]
vocab.json: 100% 798k/798k [00:00<00:00, 16.5MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 19.2MB/s]
```

tokenizer.json: 100% 2.11M/2.11M [00:00<00:00, 64.7MB/s]

special_tokens_map.json: 100% 280/280 [00:00<00:00, 28.7kB/s]

```
Transcript-based frustration score vs. the platform's own Call Sent score: Pearson c
orrelation=0.170, Spearman correlation=0.170.
Pearson and Spearman are both standard ways to measure how closely two numbers move
together,
on a scale from -1 to 1. Spearman only looks at rank order, so it is less thrown off
by outliers.

Agreement between the transcript-based score and the summary-based score: correlatio
n=0.456.
A meaningful positive correlation here means two independent pieces of text agree on
which
calls sound frustrated, which is stronger evidence than relying on the transcript al
one.
```

In [19]:
```
# Which themes carry the most frustration signal
frustration_by_theme = df.groupby("theme_label")["frustration_score"].mean().sort_v
sentiment_by_theme_check = df.groupby("theme_label")[CONFIG["sentiment_column"]].me

print("Mean frustration score by theme, higher means more frustration signal detect
print(frustration_by_theme)
print("\nMean platform Call Sent score by theme, lower means worse, shown here for
print(sentiment_by_theme_check)
```

```
Mean frustration score by theme, higher means more frustration signal detected:
theme_label
Oxygen / equipment vendor & delivery failures    0.690658
Prior authorization & equipment approval         0.673042
Pharmacy / medication access                     0.642242
Mobility equipment billing & claims              0.566572
Billing & coverage denials                       0.478060
Transportation scheduling                        0.430242
Authorized representative calls                  0.320251
Name: frustration_score, dtype: float64

Mean platform Call Sent score by theme, lower means worse, shown here for reference:
theme_label
Oxygen / equipment vendor & delivery failures   -0.821852
Authorized representative calls                 -0.620000
Billing & coverage denials                       0.098889
Transportation scheduling                        0.584615
Prior authorization & equipment approval         0.602353
Pharmacy / medication access                     0.812667
Mobility equipment billing & claims              0.853846
Name: Call Sent, dtype: float64
```

7. The primary finding: the "Synapse" vendor

While reading through the clustered transcripts in Section 4, one thing kept coming up: callers naming a specific medical equipment supplier, "Synapse". This was worth testing properly.

One wrinkle worth noting: the transcription engine spells this vendor's name a few different ways across calls. "Synapse," "Snaps," "Cnaps," and "Caps" all show up in context for what is clearly the same vendor; one row uses three different spellings in a single sentence. The count below only searches for "synapse," so it is a conservative floor on how many calls actually involve this vendor.

In [20]:
```
sent_col = CONFIG["sentiment_column"]
mentions_synapse = df[text_col].str.lower().str.contains("synapse", na=False)
syn_mean = df.loc[mentions_synapse, sent_col].mean()
other_mean = df.loc[~mentions_synapse, sent_col].mean()

t_stat, p_val = stats.ttest_ind(
    df.loc[mentions_synapse, sent_col].dropna(),
    df.loc[~mentions_synapse, sent_col].dropna(),
    equal_var=False,   # This is Welch's t-test, which does not assume the two grou
)

print(f"Calls mentioning 'Synapse': {mentions_synapse.sum()} out of {len(df)}, or {
print(f"Mean Call Sent score, Synapse calls: {syn_mean:.2f}. All other calls: {othe
print(f"Welch's t-test result: t={t_stat:.2f}, p={p_val:.4f}.")
print("A t-test checks whether two groups are actually different, not just differen
print("The p-value is the probability this gap happened by chance; the smaller it i
print("the evidence the difference is actual.")

print(f"\nStates affected: {sorted(df.loc[mentions_synapse, CONFIG['state_column']]
print(f"Plan type breakdown: {df.loc[mentions_synapse, CONFIG['lob_column']].value_
print(f"\nOf these, {(df.loc[mentions_synapse, 'ACC-1'].isin(['INCOMPLETE CALL DATA
      "'INCOMPLETE CALL DATA' by the existing system, meaning they were invisible t

print(f"\nAs a second check: does the auto-generated call summary independently agr
print(f"more frustrated too, not just the transcript? Mean summary-based frustratio
print(f"calls: {df.loc[mentions_synapse, 'summary_frustration_score'].mean():.3f}.
      f"{df.loc[~mentions_synapse, 'summary_frustration_score'].mean():.3f}.")
print("Two independent sources of text agreeing on the same pattern solidifies this
```

```
Calls mentioning 'Synapse': 18 out of 100, or 18%.
Mean Call Sent score, Synapse calls: -1.19. All other calls: 0.47.
Welch's t-test result: t=-3.90, p=0.0005.
A t-test checks whether two groups are actually different, not just different by cha
nce.
The p-value is the probability this gap happened by chance; the smaller it is, the s
tronger
the evidence the difference is actual.

States affected: ['AL', 'GA', 'IL', 'IN', 'MO', 'NC', 'SC', 'TN', 'VA']
Plan type breakdown: {'M&R': 17, 'DSNP': 1}

Of these, 5 were tagged 'INCOMPLETE CALL DATA' by the existing system, meaning they
were invisible to standard category reporting.

As a second check: does the auto-generated call summary independently agree these ca
lls are
more frustrated too, not just the transcript? Mean summary-based frustration score,
Synapse
calls: 0.242. All other calls: 0.090.
Two independent sources of text agreeing on the same pattern solidifies this findin
g.
```

Checking Other Supplier Networks

Reading through the individual Synapse transcripts manually, two other named DME suppliers, Apria and Lincare, kept showing up in similarly frustrated calls. I ran the same test on both to see if this is really one vendor's problem or a broader one.

In [21]:
```
# Testing whether the delivery/ vendor problem is unique to Synapse, or shows up un
# named suppliers too. Apria and Lincare came up repeatedly while I was reading tra
vendor_patterns = {
    "Synapse (incl. spelling variants: synops, snaps, cnaps)": r'\b(?:synapse|synop
    "Apria": r'\bapria\b',
    "Lincare": r'\bl(?:in|ynn)\s*care\b',
}
vendor_masks = {name: txt.str.contains(pat, regex=True) for name, pat in vendor_pat

vendor_checks = pd.DataFrame({
    "vendor": list(vendor_masks),
    "calls matched": [m.sum() for m in vendor_masks.values()],
    "mean Call Sent, matched": [round(df.loc[m, sent_col].mean(), 2) for m in vendo
    "mean Call Sent, rest of sample": [round(df.loc[~m, sent_col].mean(), 2) for m
})
vendor_checks
```

Out[21]:

	vendor	calls matched	mean Call Sent, matched	mean Call Sent, rest of sample
0	Synapse (incl. spelling variants: synops, snap...	19	-1.39	0.54
1	Apria	9	-0.95	0.28
2	Lincare	12	-1.17	0.35

In [22]:
```
any_named_vendor = pd.concat(vendor_masks.values(), axis=1).any(axis=1)
two_or_more = pd.concat(vendor_masks.values(), axis=1).sum(axis=1) >= 2

print(f"Calls naming at least one of these three vendors: {any_named_vendor.sum()}
      f"mean Call Sent {df.loc[any_named_vendor, sent_col].mean():.2f} versus "
      f"{df.loc[~any_named_vendor, sent_col].mean():.2f} for the rest.")
print(f"Calls naming two or more of these vendors in the same call: {two_or_more.su
      f"being switched from one to another.")
```

```
Calls naming at least one of these three vendors: 29 out of 100, mean Call Sent -1.1
5 versus 0.72 for the rest.
Calls naming two or more of these vendors in the same call: 10, usually describing b
eing switched from one to another.
```

Apria and Lincare pull almost as negative as Synapse on their own, and several calls name more than one of the three at once, usually describing being switched from one to another. Read together, this looks less like Synapse being a uniquely bad vendor and more like the DME supplier network and forced vendor switches being where the pain actually is, with Synapse simply the most frequently and specifically named one. That does not undercut the Synapse recommendation, it is still the single biggest concentration of complaints on its own, it just broadens what a root-cause conversation with vendors should probably cover.

Checking outside the keyword searches

A keyword search only catches what I thought to search for. To catch anything else, I sorted every call by its raw sentiment score and read the worst ones by hand, and ran two more general keyword checks: any mention of a denial, and any mention of a language or communication barrier with a rep.

In [23]:
```
# The 10 calls with the single lowest sentiment score in the whole sample, regardle
# or vendor, as a check on what the keyword searches above might have missed
worst = df.nsmallest(10, sent_col)[["row_num", sent_col, "ACC-1", "LOB", "Mem State
worst
```

Out[23]:

	row_num	Call Sent	ACC-1	LOB	Mem State	Call-intent auto summarization
7	9	-9.23	RX INQUIRY	DSNP	None	The caller contacted member services to report...
16	18	-5.15	None	M&R	IL	The caller is seeking assistance with ordering...
0	2	-5.10	INCOMPLETE CALL DATA	DSNP	VA	The caller is seeking clarification regarding ...
82	84	-4.90	RX REFILL DENIED	M&R	IN	The caller is seeking assistance with issues r...
5	7	-2.81	RX INQUIRY	DSNP	OH	The caller contacted member services to addres...
48	50	-2.20	None	M&R	GA	The caller is contacting to address issues rel...
22	24	-1.90	None	M&R	GA	The caller is seeking assistance to ensure tha...
27	29	-1.71	PRIOR AUTH INQUIRY	M&R	IN	The caller is frustrated with the ongoing issu...
63	65	-1.20	INCOMPLETE CALL DATA	M&R	OH	The caller is seeking assistance to order oxyg...
30	32	-1.18	INCOMPLETE CALL DATA	M&R	TN	The caller is seeking assistance with obtainin...

In [24]:
```
denial_mask = txt.str.contains(r'\b(?:denied|denial|turned down|rejected)\b', regex
language_mask = txt.str.contains(r"\b(?:understand you|understand them|understand h

print(f"Calls mentioning a denial: {denial_mask.sum()}, mean Call Sent {df.loc[deni
      f"versus {df.loc[~denial_mask, sent_col].mean():.2f} for the rest.")
print(f"Calls mentioning a language or communication barrier with a rep: {language_
      f"{df.loc[language_mask, sent_col].mean():.2f} versus {df.loc[~language_mask,
```

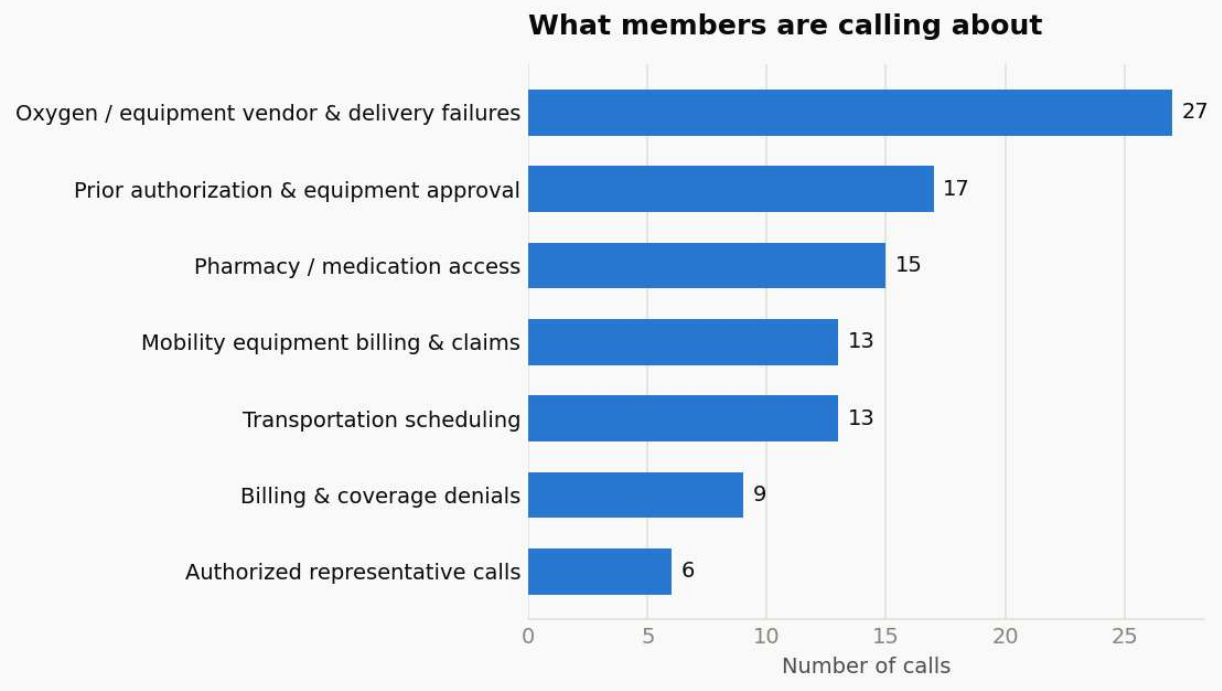
```
Calls mentioning a denial: 13, mean Call Sent 0.52 versus 0.12 for the rest.
Calls mentioning a language or communication barrier with a rep: 6, mean Call Sent -
0.38 versus 0.20 for the rest.
```

The single worst call in the entire sample is not a Synapse call. It is a caller reporting that Apria sent used or refurbished equipment while telling her it was new. The denial keyword did not turn up a broad pattern, calls mentioning a denial actually skew slightly positive, most seem to get resolved on the call itself, but one of them is worth flagging regardless of the overall average: a caller reporting that her diabetic husband's insulin pump supplies were denied directly by United Healthcare, not a vendor. That is a different category of risk than a late CPAP order and probably deserves attention on its own, independent of what the aggregate number says. The language-barrier check is a smaller signal (n=6), and it is not Synapse-specific either.

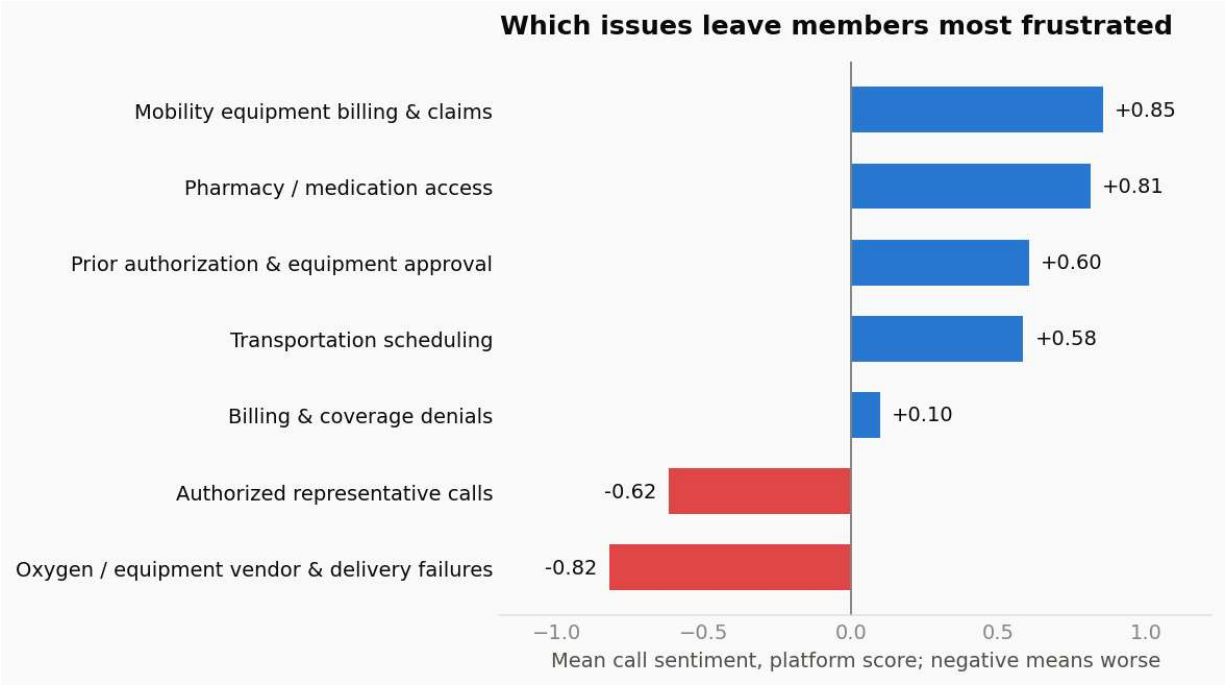
8. Charts

In [25]:
```
BLUE, RED, GRAY_MID = "#2a78d6", "#e34948", "#c3c2b7"
INK, INK_2, MUTED, GRID, SURFACE = "#0b0b0b", "#52514e", "#898781", "#e1e0d9", "#fc

plt.rcParams.update({
    "font.family": "sans-serif",
    "axes.edgecolor": GRID, "axes.labelcolor": INK_2, "text.color": INK,
    "xtick.color": MUTED, "ytick.color": INK,
    "figure.facecolor": SURFACE, "axes.facecolor": SURFACE, "savefig.facecolor": SU
})

def clean_axes(ax):
    for s in ["top", "right", "left"]:
        ax.spines[s].set_visible(False)
    ax.tick_params(length=0)
    ax.set_axisbelow(True)
```

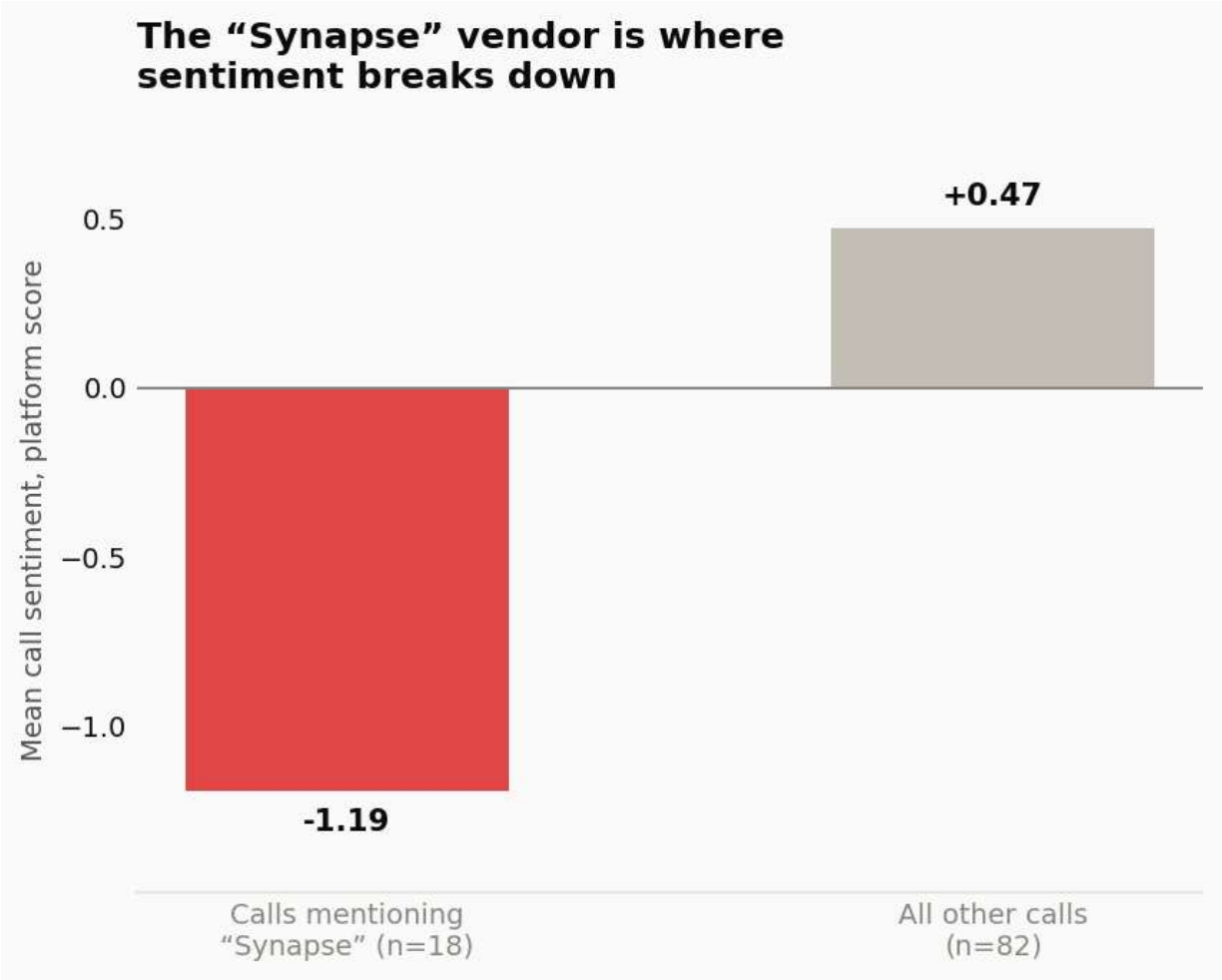
In [26]:
```
# Chart 1: call volume by theme
counts = theme_counts.sort_values()

fig, ax = plt.subplots(figsize=(8, 4.6), dpi=140)
bars = ax.barh(counts.index, counts.values, color=BLUE, height=0.6)
clean_axes(ax)
ax.set_xlabel("Number of calls")
ax.set_title("What members are calling about", loc="left", fontsize=13, fontweight=
ax.grid(axis="x", color=GRID, linewidth=0.8)

for bar, val in zip(bars, counts.values):
    ax.text(bar.get_width() + 0.4, bar.get_y() + bar.get_height() / 2, str(val), va

plt.tight_layout()
plt.show()
```



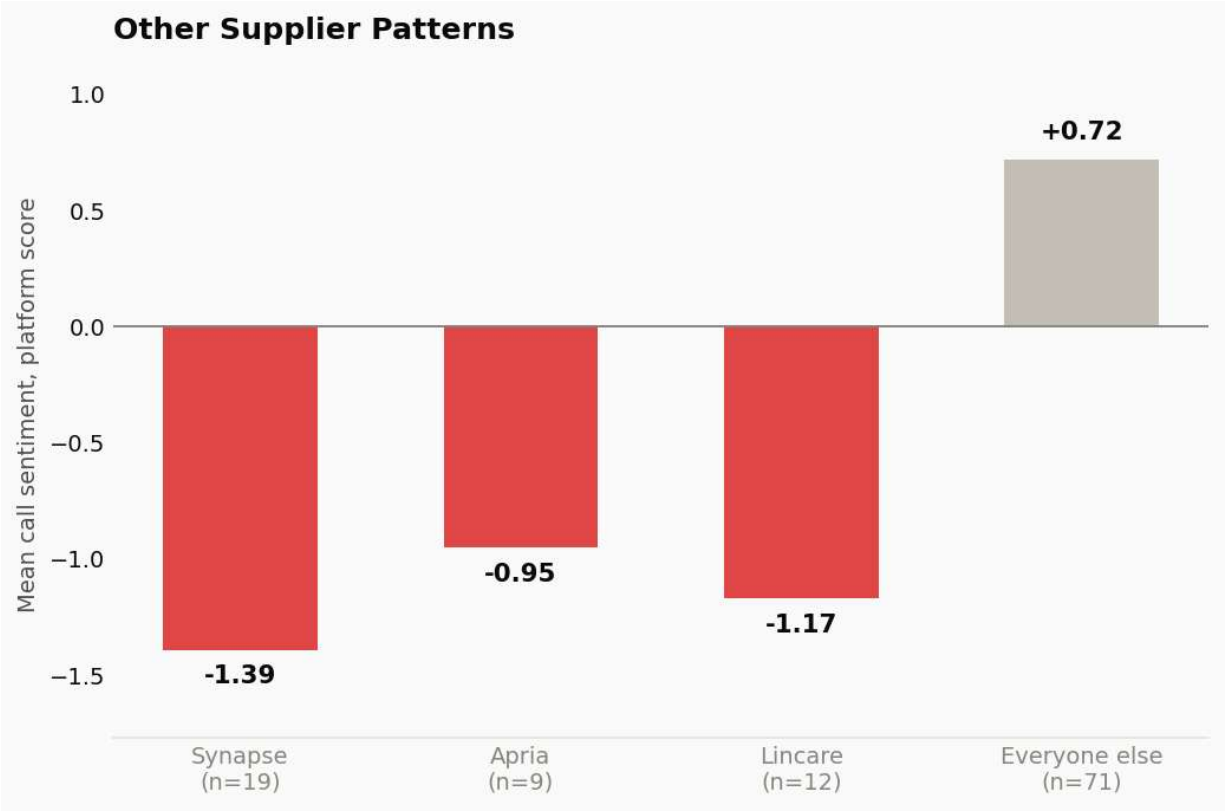
In [27]:
```
# Chart 2: sentiment by theme. Red bars are negative, blue bars are positive.
s = df.groupby("theme_label")[sent_col].mean().sort_values()
colors = [RED if v < 0 else BLUE for v in s.values]

fig, ax = plt.subplots(figsize=(8.5, 4.8), dpi=140)
bars = ax.barh(s.index, s.values, color=colors, height=0.6)
clean_axes(ax)
ax.axvline(0, color=MUTED, linewidth=1)
ax.set_xlabel("Mean call sentiment, platform score; negative means worse")
ax.set_title("Which issues leave members most frustrated", loc="left", fontsize=13,

lo, hi = s.min(), s.max()
pad = (hi - lo) * 0.22
ax.set_xlim(lo - pad, hi + pad)

for bar, val in zip(bars, s.values):
    ha = "left" if val >= 0 else "right"
    off = 0.04 if val >= 0 else -0.04
    ax.text(bar.get_width() + off, bar.get_y() + bar.get_height() / 2, f"{val:+.2f}

plt.tight_layout()
plt.show()
```



In [28]:
```
# Chart 3: Synapse calls compared with everyone else
labels = [f"Calls mentioning\n“Synapse” (n={mentions_synapse.sum()})", f"All other
vals = [syn_mean, other_mean]

fig, ax = plt.subplots(figsize=(6.2, 5.0), dpi=140)
bars = ax.bar(labels, vals, color=[RED, GRAY_MID], width=0.5)
clean_axes(ax)
ax.axhline(0, color=MUTED, linewidth=1)
ax.set_ylabel("Mean call sentiment, platform score")
ax.set_title("The “Synapse” vendor is where\nsentiment breaks down", loc="left", fo

lo_, hi_ = min(vals + [0]), max(vals + [0])
span = hi_ - lo_
ax.set_ylim(lo_ - span * 0.18, hi_ + span * 0.18)

for bar, val in zip(bars, vals):
    va = "bottom" if val >= 0 else "top"
    off = span * 0.03 if val >= 0 else -span * 0.03
    ax.text(bar.get_x() + bar.get_width() / 2, val + off, f"{val:+.2f}", ha="center

plt.tight_layout()
plt.show()
```



In [29]:
```
# Chart 4: Synapse compared with the other two named suppliers, and everyone else
labels = [f"{name.split(' (')[0]}\n(n={mask.sum()})" for name, mask in vendor_masks
labels.append(f"Everyone else\n(n={(~any_named_vendor).sum()})")
vals = list(vendor_checks["mean Call Sent, matched"]) + [round(df.loc[~any_named_ve

fig, ax = plt.subplots(figsize=(7.5, 5.0), dpi=140)
bars = ax.bar(labels, vals, color=[RED, RED, RED, GRAY_MID], width=0.55)
clean_axes(ax)
ax.axhline(0, color=MUTED, linewidth=1)
ax.set_ylabel("Mean call sentiment, platform score")
ax.set_title("Other Supplier Patterns", loc="left", fontsize=13, fontweight="bold",

lo_, hi_ = min(vals + [0]), max(vals + [0])
span = hi_ - lo_
ax.set_ylim(lo_ - span * 0.18, hi_ + span * 0.18)

for bar, val in zip(bars, vals):
    va = "bottom" if val >= 0 else "top"
    off = span * 0.03 if val >= 0 else -span * 0.03
    ax.text(bar.get_x() + bar.get_width() / 2, val + off, f"{val:+.2f}", ha="center

plt.tight_layout()
plt.show()
```



In [30]:
```
# Chart 5: how much of the raw export actually has a usable category tag
fig, ax = plt.subplots(figsize=(7, 1.6), dpi=140)
ax.barh([0], [n_tagged], color=BLUE, height=0.5)
ax.barh([0], [n_untagged], left=[n_tagged], color=GRAY_MID, height=0.5)
ax.set_xlim(0, n_tagged + n_untagged)
ax.set_yticks([]); ax.set_xticks([])
for s in ax.spines.values():
    s.set_visible(False)

pct = n_tagged / (n_tagged + n_untagged) * 100
ax.text(n_tagged / 2, 0, f"{pct:.0f}% tagged", ha="center", va="center", color="whi
ax.text(n_tagged + n_untagged / 2, 0, f"{100-pct:.0f}% untagged", ha="center", va="
ax.set_title("Existing category-tag coverage in the raw export", loc="left", fontsi

plt.tight_layout()
plt.show()
```

9. Summary and recommendations

Data quality: 2 summary-only rows were flagged. Personal information was found unredacted in 11 transcripts and redacted before the model touched it. 22% of calls have no

usable category tag, a gap large enough that it hid this report's main finding from standard reporting. The full manual-review file is described in Section 3.

Themes members are calling about: independent of the platform's own tags: equipment issues, including oxygen, prior authorization, and mobility equipment billing, make up the majority of call volume, followed by pharmacy and medication access, then transportation scheduling.

The primary finding: 18 of 100 calls, or 18%, likely an undercount as explained in Section 7, explicitly name a specific equipment vendor, "Synapse." Those calls run dramatically more negative than the rest of the sample, a difference confirmed by a standard statistical test. This is not a one-day event; it is spread across the whole sampled month and 9 states, and concentrated in Medicare & Retirement plans. Nearly a third of these calls were tagged "INCOMPLETE CALL DATA" by the existing system, and a second, independent text source (the auto-generated call summary) shows the same frustration pattern, which adds confidence this finding is worth noting.

What I would recommend:

1. Look into the Synapse vendor relationship directly. It is the clearest, statistically-supported driver of negative member experience in this sample, though as shown in Section 7, Apria and Lincare follow a similar pattern, so the review is probably better framed around the DME supplier relationship broadly rather than Synapse alone.
2. Close the category-tagging gap, currently 22% untagged, so that patterns like this one do not stay hidden from routine reporting in the future.
3. Audit the personal-information redaction step in the transcript export pipeline itself, since it should not be reaching this stage unredacted.
4. **Automation opportunity:** Use the frustration score from Section 6 to route the worst-scoring calls to quality review, not only as a one-time statistic in this analysis.